DG INTERNS HUB — TECHNICAL DOCUMENTATION

SPLUNK SPL SEARCH QUERIES — PRACTICAL SOC INVESTIGATION GUIDE

Submitted By:	**Gunjan**	**Department:** Cybersecuri
Internship:	**Cybersecurity Internship**	**Week: 4**
Submission Date:	**31 October 2026**	**Document ID:** Week_4_Sp

GUIDE INDEX & CORE SYLLABUS

1. Objective & Lab Context Setup
2. Section A: Basic SPL Queries
3. Section B: Windows Authentication Monitoring
4. Section C: Brute-Force & Spray Detection
5. Section D: SPL Commands & Functions Guide
6. Section E: Time-Based Authentication Analysis
7. Section F: SOC Forensic Investigation Queries
8. Section G: SOC Security Dashboard Queries

1. OBJECTIVE & PRACTICAL LAB CONTEXT

This technical guide documents practical Splunk Search Processing Language (SPL) queries designed for beginner SOC Analysts conducting authentication monitoring, threat detection, and forensic triage. All queries are aligned with standard Windows Security Event logs.

- **SIEM Environment:** Splunk Enterprise (Free / Developer License in Lab).
- **Data Sources:** Windows 10/11 Security Logs (WinEventLog:Security / WinLogBeat).
- **Target Event IDs:** `EventCode=4624` (Successful Logon) & `EventCode=4625` (Failed Logon).
- **Important Distinction:** `index` defines storage container; `sourcetype` defines data format/parser; `source` defines original file/stream path. Field names may vary depending on ingest agent (e.g., `TargetUserName` vs `user`, `WorkstationName` vs `host`). Placeholders like `<index_name>` indicate environment variables.

SECTION A: BASIC SPL QUERIES

A.1 — Search All Events

Objective: Broad initial query to view raw events (Lab testing only).

```
index=*
```

Explanation: `index=*` searches across all accessible Splunk indexes.
Expected Output: List of unstructured raw events from all ingested log streams.
SOC Use Case: Initial confirmation that data ingestion is active in a new lab environment.
Note: Never run this in production without time bounds due to extreme resource overhead.

A.2 — Search Events from a Specific Index

Objective: Restrict search scope to a designated index pool.

```
index=<index_name>
```

Explanation: Filters data stored strictly within the `<index_name>` container (e.g., `index=wineventlog`).
Expected Output: Log events exclusive to the specified index pool.
SOC Use Case: Speeds up query execution and enforces multi-tenant boundaries.

A.3 — Search Windows Security Logs

Objective: Narrow search to Windows OS Security audit logs.

```
index=<index_name> sourcetype="WinEventLog:Security"
```

Explanation: Combines index targeting with explicit Windows Security log sourcetype.
Expected Output: Events originating from the Windows Security Log subsystem.
SOC Use Case: Base filter for all authentication, privilege escalation, and account management investigations.

A.4 — Filter Events Using EventCode

Objective: Isolate specific Windows audit activity by ID.

```
index=<index_name> EventCode=4625
```

Explanation: Filters events where field `EventCode` equals `4625` (Failed Logon).
Expected Output: Event records representing failed authentication attempts.
SOC Use Case: Direct targeting of authentication anomalies.

A.5 — Search Events Within a Specific Time Range

Objective: Limit query scope temporally using SPL modifiers.

```
index=<index_name> EventCode=4625 earliest=-24h latest=now
```

Explanation: `earliest=-24h` sets lower time boundary to 24 hours ago; `latest=now` stops at current time.
Expected Output: Failed login events generated within the last 24 hours.
SOC Use Case: Scoping daily SOC triage shifts and incident correlation windows.

SECTION B: WINDOWS AUTHENTICATION MONITORING

B.1 & B.2 — Find Failed (4625) & Successful (4624) Login Attempts

Objective: Retrieve raw successful or failed authentication records.

```
# Failed Logins
index=<index_name> EventCode=4625

# Successful Logins
index=<index_name> EventCode=4624
```

SOC Use Case: Establishes baseline login activity for a target domain or endpoint.

B.3 — Count Total Failed Login Attempts

Objective: Aggregate overall volume of failed logins.

```
index=<index_name> EventCode=4625 | stats count as Total_Failed_Logins
```

Explanation: Passes filtered events to `stats count` to generate a single metric labeled `Total_Failed_Logins`.

B.4 & B.5 — Count Failed Logins by Username and Source IP

Objective: Categorize failures by targeted account or originating host.

```
# By Username
index=<index_name> EventCode=4625 | stats count by TargetUserName | sort - count

# By Source IP
index=<index_name> EventCode=4625 | stats count by Source_Network_Address | sort - count
```

Explanation: Groups events by field (`TargetUserName` or `Source_Network_Address`) and orders descending.
Expected Output: Ranked table showing top targeted accounts or top attacking IP addresses.

B.6 — Display Username, Source IP, and Timestamp in Table

Objective: Present essential authentication fields cleanly.

```
index=<index_name> EventCode=4625 | table _time, TargetUserName, Source_Network_Address, WorkstationName
```

Expected Output: Four-column table displaying event time, target user, source IP, and workstation.

B.7 — Identify Repeated Failed Login Attempts

Objective: Isolate accounts experiencing multiple authentication failures.

```
index=<index_name> EventCode=4625 | stats count by TargetUserName, Source_Network_Address | where count > 5 | sort - count
```

Explanation: Filters aggregated counts using `where count > 5` post-grouping.

SECTION C: BRUTE-FORCE & PASSWORD SPRAY DETECTION

C.1 & C.2 — Detect Multiple Failures per IP / Targeted Account

Objective: Spot IP addresses or user accounts under systematic attack.

```
# Single IP High-Volume Failures
index=<index_name> EventCode=4625 | stats count dc(TargetUserName) as Unique_Users by Source_Network_Address | where count > 20

# Single Account Targeted Repeatedly
index=<index_name> EventCode=4625 | stats count dc(Source_Network_Address) as Unique_IPs by TargetUserName | where count > 10
```

Explanation: `dc(...)` calculates distinct counts. High `count` with low `Unique_Users` indicates Brute Force; high `Unique_Users` with low counts per user indicates Password Spraying.

C.3 — Detect >10 Failed Logins in 15 Min per Computer (Threshold Rule)

Objective: Primary threshold alert condition for automated SOC alerting.

```
index=<index_name> EventCode=4625 earliest=-15m
| bin _time span=15m
| stats count by _time, Computer, TargetUserName, Source_Network_Address
| where count > 10
| sort - count
```

Explanation: `bin _time span=15m` groups raw events into discrete 15-minute time buckets. `stats count by _time, Computer...` aggregates failures per computer per window. `where count > 10` evaluates the threshold rule.
Expected Output: Instances where a specific computer logged more than 10 failed logins within a single 15-minute window.

C.4 — Investigate Successful Login Following Repeated Failed Attempts

Objective: Detect potentially compromised accounts (Brute-force success).

```
index=<index_name> (EventCode=4625 OR EventCode=4624)
| stats count(eval(EventCode=4625)) as Failures, count(eval(EventCode=4624)) as Successes
by TargetUserName, Source_Network_Address
| where Failures > 10 AND Successes > 0
```

Explanation: Uses conditional `eval` inside `stats` to count failures and successes simultaneously for the same user/IP pair.
SOC Severity: High / Critical. Indicates password guessing successfully breached the account.

SECTION D: CORE SPL COMMANDS & FUNCTIONS REFERENCE

Below is a quick reference table explaining key SPL commands used in SOC investigations:

- `search` : Filters raw data stream (e.g., `search EventCode=4625`).
- `where` : Filters aggregated fields post-stats (e.g., `where count > 5`).
- `table` : Selects and formats columns for display (e.g., `table _time, user, src_ip`).
- `fields` : Keeps or removes specific fields early to boost query speed (e.g., `fields + _time, EventCode`).
- `stats` : Calculates aggregate statistics (e.g., `stats count by user`).
- `eval` : Creates calculated fields or runs conditional expressions (e.g., `eval MB=Bytes/1024/1024`).
- `sort` : Sorts results in ascending or descending order (e.g., `sort - count`).
- `dedup` : Removes duplicate events based on specified fields (e.g., `dedup Source_Network_Address`).
- `rex` : Extracts custom fields using regular expressions (e.g., `rex field=_raw "user=(?<extracted_user>\w+)"`).
- `timechart` : Generates time-series data tables for graphical charting (e.g., `timechart count by EventCode`).
- `bin` : Coarsens timestamps into discrete time windows (e.g., `bin _time span=1h`).
- `top / rare` : Finds most/least frequent values of a field (e.g., `top limit=5 TargetUserName`).
- `rename` : Renames output display fields (e.g., `rename TargetUserName as User`).

SECTION E: TIME-BASED AUTHENTICATION ANALYSIS

E.1 & E.2 — Analyze Activity Over Time & Display Trends

Objective: Visualize authentication patterns using timecharts.

```
index=<index_name> (EventCode=4624 OR EventCode=4625)
| timechart span=1h count by EventCode
```

Explanation: Renders a time-series graph splitting hourly counts between 4624 (Success) and 4625 (Failure).

E.3 & E.4 — Group Events into 15-Min Windows & Spot Anomaly Spikes

Objective: Identify short bursts of intense authentication failure activity.

```
index=<index_name> EventCode=4625
| timechart span=15m count as Failed_Attempts
| eventstats avg(Failed_Attempts) as Avg_Failures, stdev(Failed_Attempts) as Std_Failures
| where Failed_Attempts > (Avg_Failures + (3 * Std_Failures))
```

Explanation: Calculates 15-minute failure rates and flags windows exceeding 3 standard deviations above the mean (statistical anomaly).

SECTION F: SOC FORENSIC INVESTIGATION WORKFLOWS

F.4 — Deep-Dive Investigation on Specific Username

Objective: Audit all authentication logs for a compromised user account.

```
index=<index_name> (EventCode=4624 OR EventCode=4625) TargetUserName="<username>"
| table _time, EventCode, WorkstationName, Source_Network_Address, Logon_Type
| sort - _time
```

SOC Workflow: Analyst checks `Logon_Type` (e.g., Type 2 = Interactive, Type 10 = RDP, Type 3 = Network) to determine attack vector.

F.5 — Deep-Dive Investigation on Specific Source IP

Objective: Identify all targets affected by a suspicious IP.

```
index=<index_name> Source_Network_Address="<source_ip>"
| stats count count(eval(EventCode=4625)) as Failures, count(eval(EventCode=4624)) as Suc
cesses, values(TargetUserName) as Targeted_Accounts by Source_Network_Address
```

SECTION G: SOC SECURITY DASHBOARD PANEL QUERIES

Use these optimized single-value and visual queries for SOC monitoring dashboards:

- **Total Failed Logins (Single Value Panel):** `index=<index_name> EventCode=4625 | stats count`
- **Total Successful Logins (Single Value Panel):** `index=<index_name> EventCode=4624 | stats count`
- **Top 5 Targeted Accounts (Bar Chart Panel):** `index=<index_name> EventCode=4625 | top limit=5 TargetUserName`
- **Top 5 Attacking IPs (Pie Chart Panel):** `index=<index_name> EventCode=4625 | top limit=5 Source_Network_Address`
- **Failed Login Trend (Area Chart Panel):** `index=<index_name> EventCode=4625 | timechart span=30m count`

CONCLUSION & ANALYST SUMMARY

Mastering these foundational SPL queries equips entry-level SOC analysts to transition from raw log viewing to structured threat hunting and automated alert tuning. Practice running these queries in your Splunk lab environment, verifying field extractions, and adjusting thresholds to match baseline network behavior.